

LINQ Cross Join in C# with Examples

In this article, I will discuss **LINQ Cross Join in C# using Method and Query Syntax** with Examples. Please read our previous article discussing [LINQ Full Outer Join in C#](#) with Examples.

What is LINQ Cross Join?

When combining two data sources (or you can say two collections) using Cross Join, each element in the first data source (i.e., first collection) will be mapped with each and every element in the second data source (i.e., second collection). So, in simple words, we can say that the cross-join produces the Cartesian Products of the collections or data sources involved in the join. This is also known as a Cartesian product, where if the first collection has m elements and the second has n elements, the result will have $m * n$ elements.

In Cross Join, we don't require the common key or property to specify the Join Condition. And moreover, there is no filtering of data. Each element of the first collection is paired with each element of the second collection. So, the total number of elements in the result set will be the product of the two data sources involved in the join. If the first data source contains five elements and the second data source contains 3 elements, then the resultant sequence will contain $(5*3)$ 15 elements.

Examples to Understand LINQ Cross Join:

Let us understand how to Implement Cross-Join in LINQ with Examples. The following Student and Subject model classes will be used to understand the LINQ Cross-Join in C#. Please create a class file named **Student.cs** and copy and paste the following code. This class has two properties and one method, returning a hard-coded collection of students, which will be one of our data sources.

```
using System.Collections.Generic;
namespace LINQCrossJoinDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Student> GetAllStudents()
        {
            return new List<Student>()
            {
                new Student { ID = 1, Name = "Preety"},
                new Student { ID = 2, Name = "Priyanka"},
                new Student { ID = 3, Name = "Anurag"},
                new Student { ID = 4, Name = "Pranaya"},
                new Student { ID = 5, Name = "Hina"}
            };
        }
    }
}
```

Next, create another class file named **Subject.cs** and copy and paste the following code. This class has two properties and one method returning a hard-coded collection of subjects, which will be another data source.

```

using System.Collections.Generic;
namespace LINQCrossJoinDemo
{
    public class Subject
    {
        public int ID { get; set; }
        public string SubjectName { get; set; }
        public static List<Subject> GetAllSubjects()
        {
            return new List<Subject>()
            {
                new Subject { ID = 1, SubjectName = "ASP.NET"},
                new Subject { ID = 2, SubjectName = "SQL Server" },
                new Subject { ID = 5, SubjectName = "Linq"}
            };
        }
    }
}

```

Example to Understand LINQ Cross Join Using Query Syntax in C#

Let us see how to implement LINQ Cross Join Using Query Syntax in C#. The following example does the Cross Join between the Student with Subject Data Sources using Query Syntax. Here, we don't need to use any "join" operator, or we don't need to use the "on" operator, and we don't need to specify the joining condition, i.e., no common property is required.

```

using System.Linq;
using System;
namespace LINQCrossJoinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Cross Join using Query Syntax
            var CrossJoinResult = from student in Student.GetAllStudents() //First Data Source
                                from subject in Subject.GetAllSubjects() //Second Data Source
                                //Projecting the Result to Anonymous Type
                                select new
                                {
                                    StudentName = student.Name,
                                    SubjectName = subject.SubjectName
                                };
            //Accessing the Elements using For Each Loop
            foreach (var item in CrossJoinResult)
            {
                Console.WriteLine($"Name : {item.StudentName}, Subject: {item.SubjectName}");
            }
        }
    }
}

```

```

    }
    Console.ReadLine();
}
}
}

```

Output:

```

Name : Preety, Subject: ASP.NET
Name : Preety, Subject: SQL Server
Name : Preety, Subject: Linq
Name : Priyanka, Subject: ASP.NET
Name : Priyanka, Subject: SQL Server
Name : Priyanka, Subject: Linq
Name : Anurag, Subject: ASP.NET
Name : Anurag, Subject: SQL Server
Name : Anurag, Subject: Linq
Name : Pranaya, Subject: ASP.NET
Name : Pranaya, Subject: SQL Server
Name : Pranaya, Subject: Linq
Name : Hina, Subject: ASP.NET
Name : Hina, Subject: SQL Server
Name : Hina, Subject: Linq

```

We have five students in the student's collection and 3 subjects in the subject's collection. In the result set, we have 15 elements, i.e., the Cartesian product of the elements involved in the joins. Instead of projecting the result to an Anonymous Type, we can also project the result to a named type. Let us understand how we can do this. For this, create a class file named **StudentSubject.cs** and copy and paste the following code. As in the result set, we need two properties, so we have created the following class with the required two properties.

```

namespace LINQCrossJoinDemo
{
    public class StudentSubject
    {
        public string StudentName { get; set; }
        public string SubjectName { get; set; }
    }
}

```

Next, we modified the Main method of the Program class. Now, instead of projecting the result to the anonymous type, we project it to the StudentSubject type.

```

using System.Linq;
using System;
namespace LINQCrossJoinDemo
{
    class Program
    {

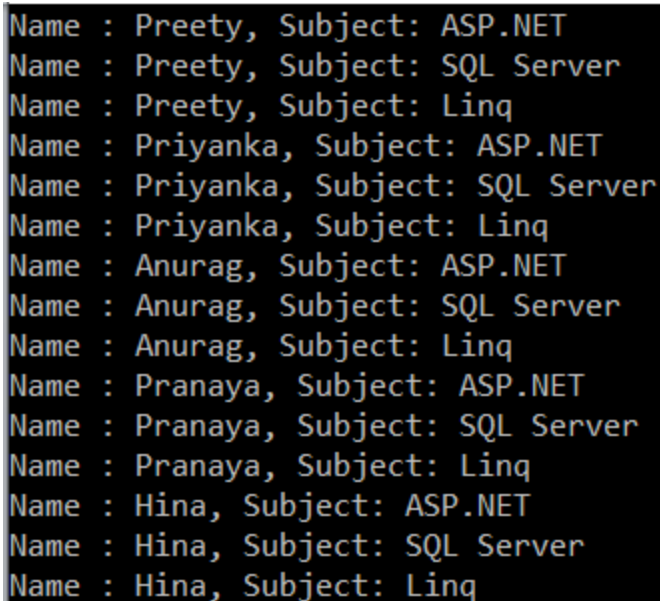
```

```

static void Main(string[] args)
{
    //Cross Join using Query Syntax
    var CrossJoinResult = from student in Student.GetAllStudents() //First Data Source
                           from subject in Subject.GetAllSubjects() //Second Data Source
                           //Projecting the Result to Anonymous Type
                           select new StudentSubject
                           {
                               StudentName = student.Name,
                               SubjectName = subject.SubjectName
                           };
    //Accessing the Elements using For Each Loop
    foreach (var item in CrossJoinResult)
    {
        Console.WriteLine($"Name : {item.StudentName}, Subject: {item.SubjectName}");
    }
    Console.ReadLine();
}
}

```

With the above changes in place, run the application, and you will get the same output as the previous example, as shown in the image below.



```

Name : Preety, Subject: ASP.NET
Name : Preety, Subject: SQL Server
Name : Preety, Subject: Linq
Name : Priyanka, Subject: ASP.NET
Name : Priyanka, Subject: SQL Server
Name : Priyanka, Subject: Linq
Name : Anurag, Subject: ASP.NET
Name : Anurag, Subject: SQL Server
Name : Anurag, Subject: Linq
Name : Pranaya, Subject: ASP.NET
Name : Pranaya, Subject: SQL Server
Name : Pranaya, Subject: Linq
Name : Hina, Subject: ASP.NET
Name : Hina, Subject: SQL Server
Name : Hina, Subject: Linq

```

Example to Understand LINQ Cross Join Using Method Syntax in C#

To implement the **LINQ Cross Join** using Method Syntax, we need to use either the **SelectMany()** method or the **Join()** method, as shown in the example below.

```

using System.Linq;

```

```

using System;
namespace LINQCrossJoinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Cross Join using SelectMany Method
            var CrossJoinResult = Student.GetAllStudents()
                .SelectMany(sub => Subject.GetAllSubjects(),
                    (std, sub) => new
                    {
                        StudentName = std.Name,
                        SubjectName = sub.SubjectName
                    });
            //Cross Join using Join Method
            var CrossJoinResult2 = Student.GetAllStudents()
                .Join(Subject.GetAllSubjects(),
                    std => true,
                    sub => true,
                    (std, sub) => new
                    {
                        StudentName = std.Name,
                        SubjectName = sub.SubjectName
                    }
                );
            foreach (var item in CrossJoinResult2)
            {
                Console.WriteLine($"Name : {item.StudentName}, Subject:
{item.SubjectName}");
            }
            Console.ReadLine();
        }
    }
}

```

It will give you the same output as the previous example, as shown in the below image.

```
Name : Preety, Subject: ASP.NET
Name : Preety, Subject: SQL Server
Name : Preety, Subject: Linq
Name : Priyanka, Subject: ASP.NET
Name : Priyanka, Subject: SQL Server
Name : Priyanka, Subject: Linq
Name : Anurag, Subject: ASP.NET
Name : Anurag, Subject: SQL Server
Name : Anurag, Subject: Linq
Name : Pranaya, Subject: ASP.NET
Name : Pranaya, Subject: SQL Server
Name : Pranaya, Subject: Linq
Name : Hina, Subject: ASP.NET
Name : Hina, Subject: SQL Server
Name : Hina, Subject: Linq
```

Instead of projecting the result to an Anonymous Type, we can also project the result to a named type. We have already created a class called StudentSubject. Let us project the result to this StudentSubject type. To do so, modify the Main method of the Program class as follows.

```
using System.Linq;
using System;
namespace LINQCrossJoinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Cross Join using SelectMany Method
            var CrossJoinResult = Student.GetAllStudents()
                .SelectMany(sub => Subject.GetAllSubjects(),
                    (std, sub) => new StudentSubject
                    {
                        StudentName = std.Name,
                        SubjectName = sub.SubjectName
                    });
            //Cross Join using Join Method
            var CrossJoinResult2 = Student.GetAllStudents()
                .Join(Subject.GetAllSubjects(),
                    std => true,
                    sub => true,
                    (std, sub) => new StudentSubject
                    {
                        StudentName = std.Name,
                        SubjectName = sub.SubjectName
                    }
                );

            foreach (var item in CrossJoinResult2)
```

```

        {
            Console.WriteLine($"Name : {item.StudentName}, Subject:
{item.SubjectName}");
        }
        Console.ReadLine();
    }
}

```

Run the application, and you will get the same output.

When Should We Use LINQ Cross Join in C#?

Here are some typical use cases for a LINQ cross-join:

- **Generating Combinations:** Cross joins are perfect for scenarios where you need to generate all possible combinations of elements from two collections. For example, pairing every color with every size when listing all product variants.
- **Data Analysis and Reporting:** Cross-joins can be used to combine data from different dimensions. For instance, combining every sales region with every product analyzes potential market opportunities.
- **Scheduling and Planning:** For scheduling applications, a Cross Join can help generate all possible combinations of times and events, like pairing every meeting room with every time slot to explore scheduling options.
- **Educational Purposes and Problem-Solving:** Cross Joins can be used for educational purposes in programming and data science, helping to illustrate concepts of combinations and permutations or to solve complex problems requiring an exploration of all possible pairings.
- **Scenario Analysis:** Cross Joins can assist in scenario analysis in business or research, where you might need to pair every potential investment option with different market conditions.
- **Matrix Operations:** Cross Joins can simulate certain types of matrix or grid operations, where you need to pair elements of two arrays or lists in a grid-like fashion.

In the next article, I will discuss the [LINQ Element Operators in C#](#) with Examples. In this article, I explain how to implement Cross Join in LINQ using **Method and Query Syntax** with Examples. I hope you enjoy this LINQ Cross Join in C# with Examples article.